Plan de Trabajo (v3) — Laboratorio de Procesos e Hilos

> Lenguaje principal: C++ (permitido por la guía: "C y/o C++ y/o Python"). Extensión opcional: Python. Cada persona tiene: **qué entra**, **qué hace (checklist)** y **qué sale**.

Nota sobre C++: el código heredado es `cpp/main.c` (se compila con `gcc` y su salida dice "Language: C"). Para que sea C++ de verdad hay que migrarlo (tarea de P1): renombrar a `main.cpp`, compilar con `g++ -std=c++17 -O0 -pthread`, y cambiar la salida a `Language: C++`. Se conservan las APIs nativas que pide la guía (`pthread_create/join`, `CreateThread`, `fork`, `CreateProcess`); `std::thread` puede usarse como variante, indicándolo en el informe.

---

0. Resumen de cambios frente al plan original

Tema	Antes	Ahora
Problema y solución	No existían	**Fase 0** con procesos e hilos por proceso definidos
Lenguaje	"C++" pero el código era C	**C++ real** (`main.cpp`, `g++`)
Procesos	Solo `multiprocessing` en Python	`fork()` / `CreateProcess` en C++
Variable medida	CPUs 1–5, 1–2 workers	**1, 2, 4, 8, 16 hilos**, 3+ repeticiones
Concurrencia vs paralelismo	No se probaba	Prueba con 1 LP y con todos los LP
Scheduler y CPU	No se observaban	`top/htop/ps -eLf/pidstat -t` y Administrador de tareas
Informe	Centrado en Amdahl	10 secciones de la guía; Amdahl como complemento
Interpretación	Sin criterio	Qué se observa / por qué / qué demuestra (25 %)

---

1. Fase 0 — Definición del problema y la solución (todos)

Regla: nadie programa ni mide hasta cerrar la Fase 0.

Decisión	Redacta	Valida
Planteamiento del problema	P4	Todos
Procesos requeridos e hilos por proceso	P1	P2, P3
Diseño experimental (matriz, repeticiones, métricas, afinidad)	P3	P1, P2
Kernel de carga, workload y banderas de compilación	P1	P2

Entra: guía del docente (PDF), código y resultados heredados. **Sale:** `docs/00_definicion.md` aprobado por los 4.

Propuesta (ajustar en la reunión):

Escenario	Procesos	Hilos por proceso	Propósito
E0 Secuencial	1	1	Línea base
E1 Multihilo	1	1, 2, 4, 8, 16	Escalamiento, concurrencia/paralelismo
E2 Multiproceso	1, 2, 4, 8	1	Hilos vs procesos
E3 Mixto	2	A: 5 hilos, B: 3 hilos	Hilos distintos por proceso (ejemplo de la guía)

Parámetros a fijar: kernel (el actual), workload C++ calibrado para ~5–10 s secuencial (inicio sugerido 2 000 000 000), workload Python ~10–20 M, banderas `-O0 -pthread` iguales en ambos SO, 5 repeticiones (mínimo 3), métricas (tiempo, CPU total %, CPU/LP, resultado numérico), afinidad (1 LP y todos).

Checklist Fase 0

- [ ] P4 redactó el planteamiento del problema.
- [ ] P1 definió procesos e hilos por proceso.
- [ ] P3 definió la matriz experimental y las métricas.
- [ ] P1 y P2 acordaron kernel y partición del rango idénticos.
- [ ] Se fijaron workloads y banderas de compilación.
- [ ] Los 4 aprobaron `docs/00_definicion.md`.

---

2. Persona 1 — Desarrollo en C++ (hilos y procesos)

ENTRA

- `docs/00_definicion.md` aprobado (escenarios, kernel, workload, banderas).
- Código heredado `cpp/main.c`.
- Guía del docente, secciones 8, 9 y 12.

HACE (checklist)

- [ ] Renombrar `main.c` → `main.cpp`; compilar con `g++ -std=c++17 -O0 -pthread`.
- [ ] Cambiar la salida a `Language: C++` y actualizar README, plan y scripts.
- [ ] Corregir advertencias (`_POSIX_C_SOURCE`, formato `PRIu64`).
- [ ] Versión secuencial.
- [ ] Versión multihilo POSIX (`pthread_create` / `pthread_join`).
- [ ] Versión multihilo Windows (`CreateThread` / `WaitForMultipleObjects`).
- [ ] Versión multiproceso Linux (`fork` / `wait`).
- [ ] Versión multiproceso Windows (`CreateProcess`).
- [ ] Escenario mixto (proceso A 5 hilos, proceso B 3 hilos).
- [ ] Hilos, procesos y workload configurables por línea de comandos.

- [ ] Calibrar el workload (secuencial ≈ 5–10 s) y comprobar que el tiempo crece con él.
- [ ] El resultado numérico es idéntico en todos los modos.
- [ ] Cada hilo imprime su procesador lógico al inicio y al fin (`sched_getcpu` / `GetCurrentProcessorNumber`).
- [ ] Salida CSV con el formato v2 (ver P3).
- [ ] Código comentado (rúbrica: 20 %).
- [ ] Compilar y probar en Linux y Windows.
- [ ] README con compilación, ejecución y ejemplos.
- [ ] Redactar la respuesta a "¿qué diferencia observó entre hilos y procesos?".

SALE

- `cpp/main.cpp` (completo, comentado, sin advertencias).
- `cpp/README.md`.
- Binarios compilables en ambos SO con las mismas banderas.
- Valor de referencia del resultado numérico por workload.
- Nota técnica hilos vs procesos para P4.
- **Va a:** P2 (kernel y partición), P3 (ejecutables y README), P4 (explicación de implementación).

Terminado cuando: P3 compila y ejecuta todos los modos solo con el README.

---

3. Persona 2 — Python (extensión opcional)

ENTRA

- `docs/00_definicion.md` aprobado.
- Kernel y partición de P1.
- Código heredado `python/`.

HACE (checklist)

- [ ] Mismo kernel y misma partición del rango que C++.
- [ ] Modos `sequential`, `threading`, `multiprocessing`.
- [ ] Workers configurables (1, 2, 4, 8, 16).
- [ ] Workload reducido para que 5 niveles de workers sean viables.
- [ ] Mismo resultado numérico que C++ para el mismo rango.
- [ ] Salida CSV formato v2.
- [ ] Funciona en Linux y Windows.
- [ ] README con dependencias y ejemplos.
- [ ] Nota técnica sobre el GIL con evidencia de las mediciones.
- [ ] Apoyar a P1 validando la equivalencia y la Fase 0.

SALE

- `python/` actualizado.
- `python/README.md`.
- Nota del GIL (`threading` vs `multiprocessing`).
- **Va a:** P3 (scripts a ejecutar), P4 (nota del GIL y comparación C++ vs Python).

Terminado cuando: P3 ejecuta todos los modos sin información adicional.

---

4. Persona 3 — Diseño experimental, pruebas y mediciones

ENTRA

- `docs/00_definicion.md` aprobado (matriz y métricas).
- Ejecutables y README de P1; scripts de P2.
- Guía del docente, secciones 7, 10, 11, 12 y 13.

HACE (checklist)

- [ ] **Exp. 1:** tabla de hardware en Windows y Linux (socket, cores por socket, threads por core, CPUs, modelo, NUMA).
- [ ] Windows: `Win32_Processor` y `Win32_ComputerSystem`. Linux: `lscpu`, `lscpu -e`, `nproc`.
- [ ] Interpretación de 4–6 líneas por equipo.
- [ ] Documentar si Linux es VM, WSL o equipo real y cuántos LP tiene.
- [ ] **Exp. 4:** ejecutar 1, 2, 4, 8 hilos (16 si el equipo lo permite).
- [ ] **Prueba de concurrencia:** afinidad a 1 LP (`taskset` / máscara en Windows).
- [ ] **Prueba de paralelismo:** todos los LP.
- [ ] Probar hilos > LP y documentar qué ocurre.
- [ ] **Exp. 6:** hilos vs procesos con el mismo N.
- [ ] Ejecutar el escenario mixto.
- [ ] **Exp. 7:** 3+ repeticiones por configuración; promedio y desviación o rango.
- [ ] Registrar CPU total y CPU/LP observado en cada corrida.
- [ ] **Exp. 5:** capturas del scheduler.
  - Linux: `top`, `htop`, `ps -eLf`, `pidstat -t -p PID 1`, `lscpu -e`.
  - Windows: Administrador de tareas (Rendimiento y Detalles), Monitor de recursos.
- [ ] Evidencia de migración de hilos (`ps -eLo pid,tid,psr,comm`, `htop` o el log de P1).
- [ ] Verificar que el resultado numérico coincide en todas las configuraciones.
- [ ] Completar `procedimiento.md` de Linux y Windows (sin campos entre corchetes).
- [ ] Documentar limitaciones (VM, compilador antiguo en Windows, carga del equipo).
- [ ] Aportar datos para las preguntas de análisis 1–10 (sección 6).

SALE

- CSV v2 con todas las mediciones individuales.
- Tablas de hardware con interpretación.
- Carpeta de capturas del scheduler y de migración.
- `procedimiento.md` completo por SO y scripts reproducibles.

- Lista de limitaciones.
- **Va a:** P4 (paquete completo de datos); P1 y P2 (reporte de errores).

Formato CSV v2

```csv
language,os,mode,processes,threads_per_process,total_workers,lp_affinity,workload,run,time_seconds,cpu_total_pct,resul
C++,linux,threads,1,8,8,all,2000000000,1,5.231,780,15594732105878822272
C++,linux,threads,1,8,8,1,2000000000,1,40.882,100,15594732105878822272
```

Terminado cuando: P4 arma todas las tablas y gráficas sin volver a ejecutar nada ni pedir datos.

---

5. Persona 4 — Planteamiento, informe e interpretación

ENTRA

- Guía del docente (estructura, rúbrica y preguntas).
- Definición de la Fase 0 (la redacta ella/él).
- Paquete de datos de P3; explicaciones de P1 y P2.

HACE (checklist)

- [ ] Redactar el planteamiento del problema y coordinar la Fase 0.
- [ ] Avanzar introducción y marco conceptual (proceso, hilo, CPU, socket, núcleo, LP, scheduler, concurrencia, paralelismo).
- [ ] Preparar plantillas de tablas y gráficas mientras esperan a P1–P3.
- [ ] Caracterización del equipo (desde P3).
- [ ] Implementación: explicar hilos y procesos en C++ (y Python).
- [ ] Experimentos: procedimiento, configuraciones y herramientas.
- [ ] Resultados: tablas R1–R3, promedio y desviación.

- [ ] Gráfica de tiempo vs número de hilos.
- [ ] Gráfica de CPU vs número de hilos.
- [ ] Tabla comparativa Windows/Linux (sección 16 de la guía).
- [ ] Análisis: cada evidencia con ¿qué se observa? ¿por qué ocurre? ¿qué demuestra?
- [ ] Responder las 10 preguntas de análisis.
- [ ] Complementos: C++ vs Python, speedup, eficiencia, Amdahl.
- [ ] Conclusiones: qué ocurrió, por qué y relación con hardware y scheduler.
- [ ] Referencias.
- [ ] Verificación cruzada: cada cifra del informe coincide con los CSV.

SALE

- Informe final (10 secciones) con anexos.
- Gráficas y tablas.
- Respuestas a las 10 preguntas.
- Referencias.
- **Va a:** el docente.

Terminado cuando: todas las cifras tienen respaldo en el repositorio y cada evidencia está interpretada.

---

6. Matriz de entradas y salidas

De → A	Qué se entrega
Todos → Fase 0	Aprobación de `docs/00_definicion.md`
P1 → P2	Kernel, partición y resultado de referencia
P1 → P3	Ejecutables, README, log de LP por hilo
P1 → P4	Explicación de la implementación y nota hilos vs procesos
P2 → P3	Scripts de Python y README
P2 → P4	Nota del GIL y comparación con C++
P3 → P1, P2	Errores y ajustes
P3 → P4	CSV v2, hardware, capturas, procedimiento, limitaciones
P4 → Docente	Informe final

7. Preguntas de análisis: quién aporta qué

#	Pregunta	Datos	Redacta
1	CPUs físicas, sockets, núcleos y LP	P3	P4
2	¿LP coincide con núcleos?	P3	P4
3	Evidencia de concurrencia	P3 (1 LP)	P4
4	Evidencia de paralelismo	P3 (CPU total, LP por hilo)	P4
5	Hilos > LP	P3	P4
6	¿Baja el tiempo linealmente?	P3	P4
7	¿Migran los hilos? ¿Cómo se observa?	P1 + P3	P4
8	Hilos vs procesos	P1	P4
9	Limitaciones	P3	P4
10	Windows vs Linux	P3	P4

8. Estructura del informe y rúbrica

Sección	Autor principal
1. Introducción	P4
2. Marco conceptual	P4
3. Caracterización del equipo	P3 → P4
4. Implementación	P1 (+P2) → P4
5. Experimentos	P3 → P4
6. Resultados	P3 → P4
7. Análisis	P4
8. Comparación Windows/Linux	P3 + P4
9. Conclusiones	P4
10. Referencias	P4

Criterio	Peso	Dónde se gana
Fundamentos	15 %	Marco conceptual (P4)
Implementación	20 %	Código funcional y comentado (P1, P2)
Diseño experimental	15 %	Fase 0 y matriz (P3)
Medición	15 %	Tablas, repeticiones, gráficas (P3, P4)
Interpretación	**25 %**	Qué/por qué/qué demuestra (P4 con datos de P3)
Presentación	10 %	Informe ordenado (P4)

9. Orden de trabajo

```text
Fase 0 (todos)
   ├──> P1 (C++) ─┐
   ├──> P2 (Py) ──┼──> P3 (pruebas) ──> P4 (informe final)
   ├──> P3 empieza ya: hardware + herramientas de scheduler
   └──> P4 empieza ya: introducción, marco conceptual, plantillas
```